

Prompt:

8/17/2026, 10:12:59 PM

lanjut Tahap Z

Response:

8/17/2026, 10:13:01 PM

Tahap Z — Concrete Model Specification & Implementation Blueprint

Baik. Tahap Z adalah transisi dari desain matematis ke sistem yang benar-benar dapat diimplementasikan.

Sampai Tahap Y kita sudah mempunyai:

$$F_t \rightarrow s_t \rightarrow (\hat{\mu}_t, \hat{\Sigma}_t) \rightarrow H_t \rightarrow x_t^* \rightarrow w_t^* \rightarrow q_t^* \rightarrow O_t \rightarrow s_{t+1}.$$

Sekarang kita harus menjawab pertanyaan yang jauh lebih konkret:

"Data apa yang masuk? Diproses bagaimana? Persamaan persisnya apa? Output tiap modul apa? Dan di mana QUBO/Ising/VQE/QAOA benar-benar digunakan?"

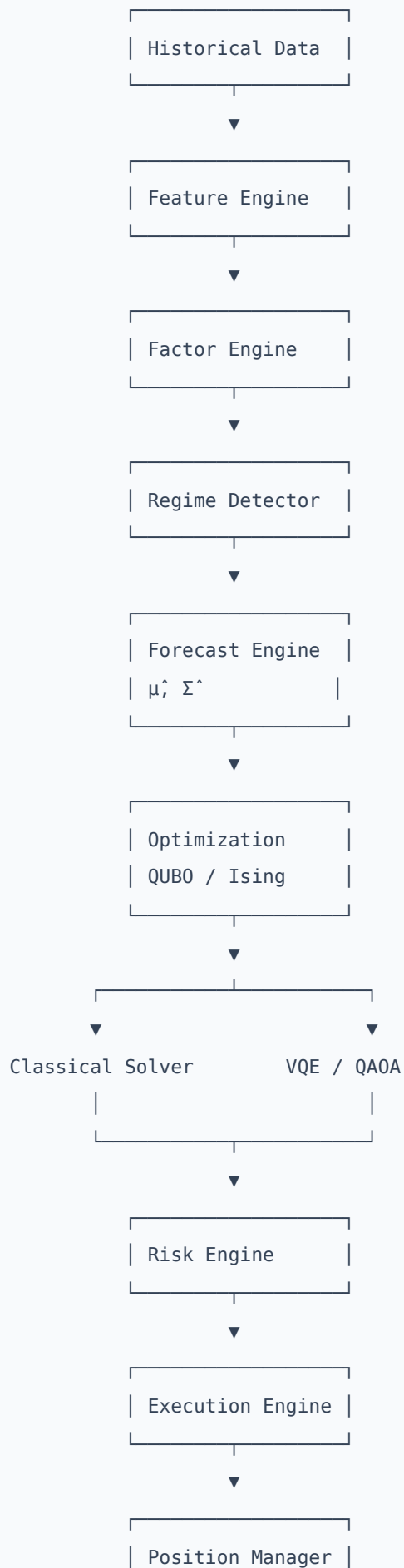
Saya akan membuat Tahap Z sebagai **blueprint teknis versi 1.0**. Kita jangan langsung coding sebelum struktur ini konsisten.

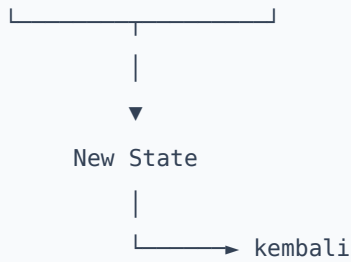
Z.1. Prinsip desain utama

Saya ingin kita memisahkan sistem menjadi **10 layer**:



Secara arsitektur:





Z.2. Jangan langsung mulai dengan 500 aset

Ini keputusan desain yang penting.

Versi pertama sebaiknya menggunakan **small controlled universe**.

Misalnya untuk crypto:

$$A_{crypto} = \{BTC, ETH, SOL, BNB, XRP, \dots\}$$

sekitar:

$$N = 8 - 20.$$

Untuk saham:

$$A_{equity} = \{AAPL, MSFT, NVDA, AMZN, \dots\}.$$

Dan factor:

$$F = \{SPX, NDX, DXY, Gold, VIX, US10Y\}.$$

Tujuannya bukan membuat sistem terbesar.

Tujuannya:

make the mathematics auditable.

Z.3. Multi-asset universe

Saya menyarankan kita mendefinisikan tiga kategori.

Tier 1 — Target assets

Asset yang benar-benar boleh ditradingkan:

$$A_T.$$

Contoh:

$$A_T = \{SOL, XRP, ADA, \dots\}.$$

Tier 2 — Market leaders

Asset yang digunakan sebagai explanatory factors:

$$A_M = \{BTC, ETH\}.$$

Tier 3 — Macro factors

$$A_F = \{DXY, Gold, VIX, SPX, NDX, US10Y\}.$$

Dengan demikian:

$$A = A_T \cup A_M \cup A_F.$$

Z.4. Timeframe

Kita perlu membedakan:

Data resolution

Misalnya:

$$\Delta t = 1h.$$

Decision interval

Misalnya:

$$\Delta t_D = 1h.$$

Execution resolution

Misalnya:

$$\Delta t_E = 1m.$$

Ini sangat penting.

Kita tidak boleh melakukan:

prediksi 1 jam $\rightarrow$ eksekusi menggunakan harga close 1 jam

tanpa memodelkan bagaimana order benar-benar terisi.

Jadi:

$$1h \rightarrow \text{decision}$$

sedangkan:

$$1m \rightarrow \text{execution.}$$

Z.5. Dataset utama

Untuk setiap asset i , kita simpan:

$$D_{i,t} = [O, H, L, C, V].$$

Dengan:

- O : open,
- H : high,
- L : low,
- C : close,
- V : volume.

Untuk order book:

$$OB_t = [B_1, A_1, Q_{B1}, Q_{A1}, \dots].$$

Z.6. Return

Kita gunakan log return:

$$r_{i,t} = \ln \frac{P_{i,t}}{P_{i,t-1}}.$$

Untuk multi-asset:

$$\mathbf{r}_t = [r_{1,t}, r_{2,t}, \dots, r_{N,t}]^\top.$$

Z.7. Feature vector

Untuk setiap asset:

$$X_{i,t} = [r_t, r_{t-1}, \dots, r_{t-L}, Vol_t, Momentum_t, ATR_t, RSI_t, \dots].$$

Tetapi saya menyarankan **jangan memasukkan puluhan indikator teknikal sejak awal**.

Kita ingin mengetahui kontribusi setiap kelompok informasi.

Maka feature groups:

$$X_t = [X_t^{price}, X_t^{volume}, X_t^{factor}, X_t^{orderflow}, X_t^{regime}].$$

Z.8. Factor matrix

Misalkan kita mempunyai:

$$K$$

factor.

Maka:

$$F_t = \begin{bmatrix} f_{1,t} \\ f_{2,t} \\ \vdots \\ f_{K,t} \end{bmatrix}.$$

Contoh:

$$F_t = [r_{BTC}, r_{ETH}, r_{DXY}, r_{Gold}, r_{SPX}, r_{VIX}, r_{US10Y}]^T.$$

Z.9. Standardisasi

Karena skala masing-masing factor berbeda:

$$z_{k,t} = \frac{f_{k,t} - \mu_{k,t}^{train}}{\sigma_{k,t}^{train}}.$$

Penting:

$$\mu_{k,t}^{train}$$

dan:

$$\sigma_{k,t}^{train}$$

harus dihitung **hanya menggunakan data yang tersedia sampai waktu training tersebut.**

Tidak boleh:

fit scaler pada seluruh dataset.

Itu data leakage.

Z.10. Factor model

Kita gunakan:

$$r_{i,t+1} = \alpha_{i,t} + \beta_{i,t}^\top F_t + \epsilon_{i,t+1}.$$

Secara matrix:

$$\mathbf{r}_{t+1} = \alpha_t + B_t F_t + \epsilon_{t+1}.$$

Z.11. Estimasi beta

Untuk versi awal:

$$\hat{\beta}_i = (X^\top X)^{-1} X^\top y_i.$$

Namun untuk rolling system:

$$\hat{\beta}_{i,t} = \text{OLS}(r_{i,t-L:t}, F_{t-L:t}).$$

Jadi beta berubah terhadap waktu.

Z.12. Expected return

Kita memperoleh:

$$\hat{\mu}_t = \hat{\alpha}_t + \hat{B}_t F_t.$$

Tetapi saya menyarankan kita tidak berhenti di sini.

Kita dapat menambahkan signal component:

$$\hat{\mu}_t = \hat{\mu}_t^{factor} + \hat{\mu}_t^{signal}.$$

Misalnya:

$$\hat{\mu}_t = B_t F_t + g_\theta(X_t).$$

Dengan demikian:

- factor model memberikan economic structure,
 - signal model menangkap nonlinear residual behavior.
-

Z.13. Versi pertama tidak perlu deep learning

Ini keputusan penting.

Untuk baseline:

bisa berupa:

- linear regression,
- ridge,
- elastic net,
- random forest,
- gradient boosting.

Baru setelah baseline valid, kita bandingkan dengan model neural.

Karena kalau langsung menggunakan deep learning dan hasil bagus:

kita tidak tahu apakah keuntungan berasal dari model atau leakage/overfitting.

Z.14. Covariance matrix

Residual:

$$\hat{\epsilon}_t = r_t - \alpha_t - \hat{B}_t F_t.$$

Covariance factor:

$$\Sigma_F = \text{Cov}(F).$$

Residual covariance:

$$D = \text{Cov}(\epsilon).$$

Kemudian:

$$\hat{\Sigma}_t = \hat{B}_t \Sigma_{F,t} \hat{B}_t^\top + D_t.$$

Z.15. Shrinkage

Covariance matrix short-term biasanya noisy.

Maka kita dapat menggunakan:

$$\Sigma_{shrink} = \rho T + (1 - \rho) \Sigma.$$

dengan:

- T : target covariance,
- ρ : shrinkage intensity.

Ini penting karena optimizer sangat sensitif terhadap covariance.

Z.16. Regime engine

Kita definisikan:

$$z_t \in \{1, \dots, K\}.$$

Feature regime:

$$Y_t = [\sigma_t, VIX_t, Trend_t, Breadth_t, Liquidity_t, \dots].$$

Versi pertama dapat menggunakan:

$$z_t = \text{HMM}(Y_t).$$

Misalnya:

$$K = 3.$$

Z.17. Tiga regime awal

Saya menyarankan:

$$z_t = \begin{cases} 1 & \text{Calm} \\ 2 & \text{Trending} \\ 3 & \text{Stress.} \end{cases}$$

Kemudian:

$$\lambda_t = \lambda(z_t).$$

Misalnya secara konseptual:

$$\lambda_{\text{stress}} > \lambda_{\text{calm}}.$$



Z.18. Regime juga mengubah expected return

Bukan hanya risk aversion.

Kita dapat menggunakan:

$$\mu_t = \mu^{(z_t)} + B^{(z_t)} F_t.$$

Dan:

$$\Sigma_t = B^{(z_t)} \Sigma_F^{(z_t)} B^{(z_t)\top} + D^{(z_t)}.$$

Ini menjadikan model **regime conditional**.

Z.19. Sekarang masuk ke optimizer

Kita mendefinisikan binary variable:

$$x_i = \begin{cases} 1, & \text{asset } i \text{ selected} \\ 0, & \text{otherwise.} \end{cases}$$

Portfolio:

$$x \in \{0, 1\}^N.$$

Z.20. Objective

Kita mulai dengan:

$$H(x) = -\mu^\top x + \lambda x^\top \Sigma x.$$

Interpretasi:

$$-\mu^\top x$$

berusaha memaksimalkan expected return.

Sedangkan:

$$\lambda x^T \Sigma x$$

menghukum risk.

Z.21. Capital constraint

Misalkan setiap asset mempunyai allocation weight w_i .

Constraint:

$$\sum_i w_i x_i \leq 1.$$

Penalty:

$$P_c \left(\sum_i w_i x_i - 1 \right)^2.$$

Z.22. Cardinality

Jika maksimum asset:

$$K_{max},$$

maka:

$$\sum_i x_i \leq K_{max}.$$

Penalty:

$$P_K \left(\sum_i x_i - K_{max} \right)^2.$$

Jika kita ingin **tepat** K :

$$\boxed{P_K \left(\sum_i x_i - K \right)^2}.$$

Z.23. Turnover

Misalkan:

$$x_i^{old}.$$

Biaya:

$$C_{turn} = c_i |x_i - x_i^{old}|.$$

Karena binary:

$$|x_i - x_i^{old}| = x_i + x_i^{old} - 2x_i x_i^{old}.$$

Sehingga:

$$C_{turn} = \sum_i c_i (x_i + x_i^{old} - 2x_i x_i^{old}).$$

Ini quadratic.

Sangat cocok untuk QUBO.

Z.24. Final QUBO

Maka versi pertama:

$$\begin{aligned} H_t(x) = & -\mu_t^\top x + \lambda_t x^\top \Sigma_t x \\ & + \sum_i c_i |x_i - x_i^{old}| \\ & + P_K \left(\sum_i x_i - K \right)^2 \\ & + P_C \left(\sum_i w_i x_i - C \right)^2. \end{aligned}$$

Ini adalah **QUBO** utama sistem kita.

Z.25. Constraint tambahan

Nanti kita dapat menambahkan:

Maximum sector exposure

$$\sum_{i \in \text{sector } j} x_i \leq K_j.$$

Maximum correlated assets

$$x_i + x_j \leq 1.$$

Minimum liquidity

$$x_i = 0 \quad \text{if} \quad ADV_i < ADV_{min}.$$

Maximum leverage

$$\sum_i |w_i| \leq L.$$

Semua ini dapat kita ubah menjadi binary constraints/penalties jika diperlukan.

Z.26. Penalty coefficient

Ini bagian yang sangat sering salah.

Tidak boleh asal:

$$P = 1.$$

Secara prinsip:

$$P > \text{maximum possible objective gain from violating constraint.}$$

Misalnya:

$$P_K > \max_x |\mu^\top x|.$$

Namun kita harus hati-hati karena penalty terlalu besar juga membuat landscape numeriknya buruk.
Maka nanti kita lakukan:

$$P \in \{P_1, P_2, \dots, P_m\}$$

dan melakukan sensitivity analysis.

Z.27. QUBO $\rightarrow$ Ising

Gunakan:

$$x_i = \frac{1 - Z_i}{2}.$$

Secara umum:

$$H(x) \rightarrow H(Z).$$

Kemudian:

$$H(Z) = C + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Quantum Hamiltonian:

$$\hat{H} = CI + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Z.28. Jumlah qubit

Jika:

$$N = 10$$

asset binary:

$$n_q = 10.$$

Jika kita menggunakan long/short encoding:

$$x_i^+, x_i^-,$$

maka:

$$n_q = 2N.$$

Untuk tahap awal saya sangat menyarankan:

$$N = 8-12$$

supaya kita dapat membandingkan:

$$Exact \leftrightarrow Classical \leftrightarrow VQE \leftrightarrow QAOA.$$

Z.29. Exact solver adalah wajib

Jangan langsung membandingkan:

VQE vs market.

Kita harus memiliki ground truth.

Untuk:

$$N = 10,$$

jumlah kemungkinan:

$$2^{10} = 1024.$$

Kita dapat brute force seluruh state.

Sehingga:

$$x_{exact}^* = \arg \min_{x \in \{0,1\}^{10}} H(x).$$

Ini memberi:

$$E_{exact}.$$

Z.30. Quantum approximation ratio

Jika QAOA/VQE menghasilkan:

$$E_Q,$$

sedangkan optimum:

$$E^*,$$

kita dapat mengukur:

$$\boxed{Gap = E_Q - E^*}.$$

Untuk minimization:

$$Gap \geq 0.$$

Relative gap:

$$\boxed{Gap_{rel} = \frac{E_Q - E^*}{|E^*|}}.$$

Ini jauh lebih ilmiah daripada sekadar:

"QAOA menghasilkan profit lebih tinggi."

Z.31. VQE baseline

VQE:

$$|\psi(\theta)\rangle = U(\theta)|0\rangle^{\otimes N}.$$

Energy:

$$E(\theta) = \langle \psi(\theta) | \hat{H}_t | \psi(\theta) \rangle.$$

Optimasi:

$$\theta^* = \arg \min_{\theta} E(\theta).$$

Z.32. Ansatz

Untuk baseline:

$$U(\theta) = U_{ent} U_{rot}.$$

Misalnya:

$$R_y(\theta_i)$$

dan entanglement:

$$CX_{i,i+1}.$$

Kita bisa menggunakan EfficientSU2 untuk baseline.

Namun jangan lupa:

ansatz bukan bagian dari market model.

Ia adalah computational mechanism untuk mendekati ground state.

Z.33. QAOA baseline

Untuk QAOA:

$$|\gamma, \beta\rangle = \prod_{p=1}^P e^{-i\beta_p H_M} e^{-i\gamma_p H_C} |+\rangle^{\otimes N}.$$

Objective:

$$E(\gamma, \beta) = \langle H_C \rangle.$$

Z.34. Classical benchmark

Minimal kita punya:

Exact

SLSQP/continuousrelaxation

MILP/classicalQUBOsolver

VQE

QAOA

Tidak semua harus digunakan pada versi pertama, tetapi exact solver **harus** ada.

Z.35. Dari binary selection ke actual portfolio

Ini juga penting.

Misalkan QUBO menghasilkan:

$$x^* = [1, 0, 1, 0, 1].$$

Itu **belum** berarti:

$$w = [0.33, 0, 0.33, 0, 0.33].$$

Kita perlu allocation layer.

Untuk selected assets:

$$S = \{i : x_i^* = 1\}.$$

Kemudian solve:

$$w^* = \arg \max_w [w^T \mu - \lambda w^T \Sigma w]$$

subject to:

$$w_i = 0 \quad i \notin S.$$

Z.36. Jadi QUBO bukan necessarily optimizer seluruh weight

QUBO kita dapat digunakan sebagai:

asset selection layer.

Sedangkan continuous optimizer digunakan untuk:

capital allocation layer.

Ini menurut saya desain yang jauh lebih kuat.

Z.37. Dari weight ke risk budget

Setelah:

$$w^*$$

didapat, kita hitung:

$$Risk_{portfolio} = \sqrt{w^{*\top} \Sigma w^*}.$$

Kemudian scaling:

$$w^{scaled} = w^* \frac{R_{target}}{Risk_{portfolio}}.$$

Dengan batas:

$$\|w^{scaled}\|_1 \leq L_{max}.$$

Z.38. Dari portfolio weight ke quantity

Jika capital:

$$C_t,$$

maka dollar allocation:

$$A_i = C_t w_i.$$

Quantity:

$$q_i = \frac{A_i}{P_i}.$$

Kemudian risk constraint:

$$q_i |P_i - SL_i| \leq R_i.$$

Sehingga:

$$q_i = \min \left[\frac{C_t w_i}{P_i}, \frac{R_i}{|P_i - SL_i|} \right].$$

Ini adalah quantity final.

Z.39. Stop-loss engine

Untuk versi awal:

$$SL_i = P_i - k_\sigma \sigma_i$$

untuk long.

Untuk short:

$$SL_i = P_i + k_\sigma \sigma_i.$$

Lebih tepat jika:

$$\sigma_i$$



dikonversi menjadi volatility pada horizon trade.

Misalnya:

$$\sigma_{\Delta t} = \sigma_{daily} \sqrt{\frac{\Delta t}{1day}}.$$

Z.40. TP engine

Gunakan:

$$TP_i = P_i + \rho_i(P_i - SL_i)$$

untuk long.

$$TP_i = P_i - \rho_i(SL_i - P_i)$$

untuk short.

Dengan:

$$\rho_i = f(\mu_i, \sigma_i, regime, liquidity).$$

Z.41. Expected trade value

Kita hitung:

$$EV_i = p_i^{win} R_i - (1 - p_i^{win}) L_i - Cost_i.$$

Trade hanya boleh diteruskan jika:

$$EV_i > 0.$$

Ini menjadi **final gate** setelah optimizer.

Z.42. Execution engine

Execution engine menerima:

TradePlan

└─ symbol

└─ side

└─ quantity

└─ entry

└─ stop_loss

└─ take_profit

└─ order_type

└─ expiry

Kemudian menentukan:

Market

atau:

Limit.

Z.43. Limit order model

Kita harus mensimulasikan:

$P(Fill)$.

Jangan menggunakan:

"kalau candle menyentuh entry maka order filled."

Itu terlalu optimistik.

Minimal kita perlu mempertimbangkan:

distance, spread, volume, queue, intrabar path.

Z.44. Execution cost

Actual return:

$$R_{actual} = R_{gross} - Fee - Spread - Slippage - Impact.$$

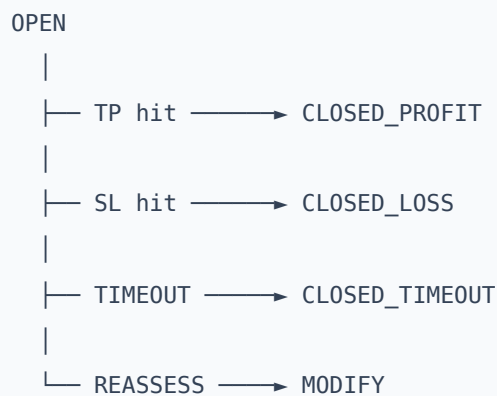


Sehingga:

$$R_{net} = R_{gross} - C_{execution}.$$

Z.45. Position manager

Setelah filled:



Z.46. Reassessment

Setiap:

$$\Delta t_R$$

kita hitung:

$$EV_{hold}.$$

Kemudian:

$$EV_{hold}$$

dibandingkan dengan:

$$EV_{exit}.$$

Jika:

$$EV_{hold} > EV_{exit},$$

hold.

Jika:

$$EV_{exit} > EV_{hold},$$

exit.

Z.47. Break-even rule

Jika:

$$P nL_t > \theta_{BE} R_i,$$

maka:

$$SL_t \rightarrow Entry.$$

Namun kita dapat menggunakan:

$$SL_t = Entry + \epsilon$$

untuk long.

Z.48. Trailing stop

Jika momentum tetap kuat:

$$SL_t = \max (SL_{t-1}, P_t - k\sigma_t).$$

Ini memastikan:

$$SL_t$$

hanya bergerak ke arah yang mengurangi downside.

Z.49. Dynamic TP

Misalkan:

$$S_t = \frac{\dot{\mu}_t}{\dot{\sigma}_t}.$$

Jika:

$$S_t > S_{high},$$

kita dapat mempertahankan/menaikkan TP.

Jika:

$$S_t < S_{low},$$

kita kurangi target atau exit.

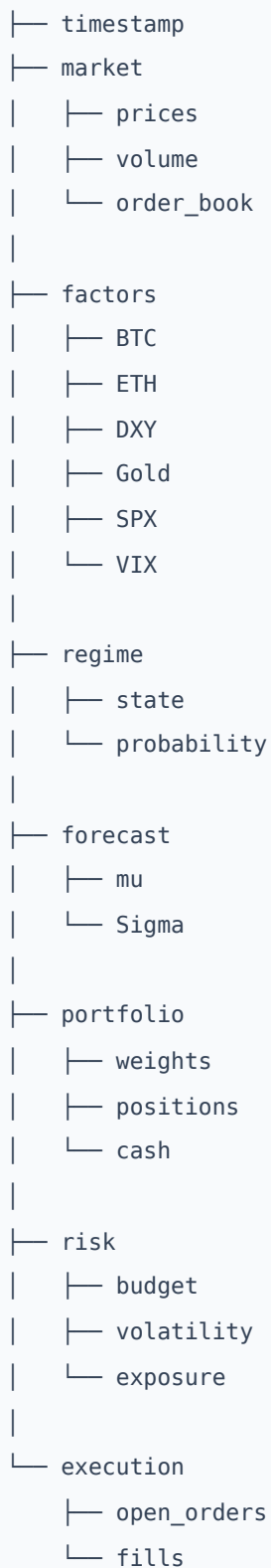
Jadi:

$$TP_{t+1} = f(TP_t, S_t, z_t, PnL_t).$$

Z.50. Full state object

Secara implementasi, saya menyarankan satu objek konseptual:

State



Z.51. Pipeline data

Secara computational:

Data → *FeatureData* → *FactorData* → *RegimeData* → *ForecastData* → *QUBO* → *Solution* → *TradePlan* –



Ini akan sangat membantu ketika nanti kita mulai membuat repository.

Z.52. Struktur repository yang saya sarankan

Saya akan mengubah proyek kita menjadi kira-kira:

```
quant-system/
|
├─ data/
|   ├── raw/
|   ├── processed/
|   └─ features/
|
├─ config/
|   ├── assets.yaml
|   ├── factors.yaml
|   └─ strategy.yaml
|
├─ src/
|   ├── data/
|   |   ├── loader.py
|   |   └─ cleaner.py
|   |
|   ├── features/
|   |   └─ feature_engine.py
|   |
|   ├── factors/
|   |   └─ factor_model.py
|   |
|   ├── regime/
|   |   └─ regime_model.py
|   |
|   ├── forecast/
|   |   └─ return_model.py
|   |
|   ├── optimization/
|   |   ├── qubo.py
|   |   ├── ising.py
|   |   ├── exact.py
|   |   ├── classical.py
|   |   ├── vqe.py
|   |   └─ qaoa.py
|   |
|   ├── risk/
|   |   ├── sizing.py
|   |   ├── stop_loss.py
|   |   └─ risk_manager.py
|   |
|   |
```

```
| | └─ execution/
| |   └─ order.py
| |   └─ fill_model.py
| |   └─ slippage.py
| |
| | └─ portfolio/
| |   └─ allocator.py
| |
| | └─ backtest/
| |   └─ engine.py
| |
| └─ evaluation/
|   └─ metrics.py
|   └─ attribution.py
|
└─ notebooks/
└─ experiments/
└─ tests/
└─ README.md
```

Z.53. Sangat penting: jangan membuat `quantum.py` sebagai pusat sistem

Quantum hanyalah:

Optimization Backend.

Struktur:

```
OptimizationProblem
|
└─ ExactSolver
└─ ClassicalSolver
└─ VQE
└─ QAOA
```

Semua menerima **problem yang sama**.

Ini memungkinkan kita melakukan eksperimen yang benar-benar fair.

Z.54. Interface solver

Secara konseptual:

```
solve(problem)
  |
  ├── exact
  ├── classical
  ├── vqe
  └── qaoa
```

Output:

```
Solution
├── bitstring
├── objective_value
├── feasible
├── runtime
└── metadata
```

Dengan begitu:

$Solution_{VQE}$

dan:

$Solution_{QAOA}$

memiliki format yang sama.

Z.55. Backtest engine

Backtest harus berjalan secara chronological:

$$t_0 \rightarrow t_1 \rightarrow t_2 \rightarrow \dots \rightarrow t_T.$$

Pada setiap t :

1. update data,
2. update feature,

3. estimate factor,
 4. detect regime,
 5. estimate μ, Σ ,
 6. build QUBO,
 7. solve,
 8. generate order,
 9. simulate execution,
 10. update position,
 11. update equity.
-

Z.56. Tidak boleh ada future leakage

Pada waktu:

$$t,$$

model hanya boleh menggunakan:

$$D_{\leq t}.$$

Bukan:

$$D_{> t}.$$

Dengan kata lain:

$$\hat{\theta}_t = \text{Fit}(D_{\leq t}).$$

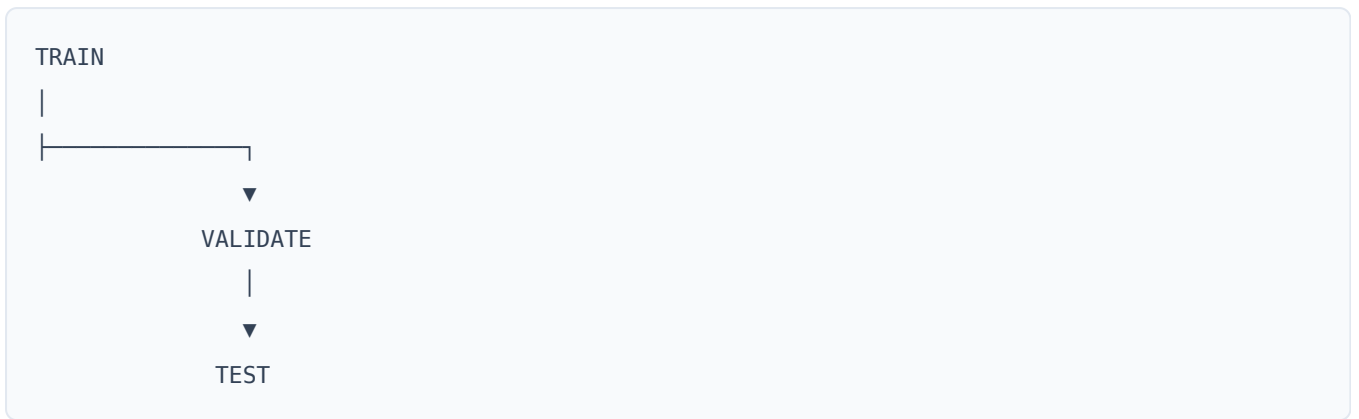
Bukan:

$$\text{Fit}(D_{0:T}).$$

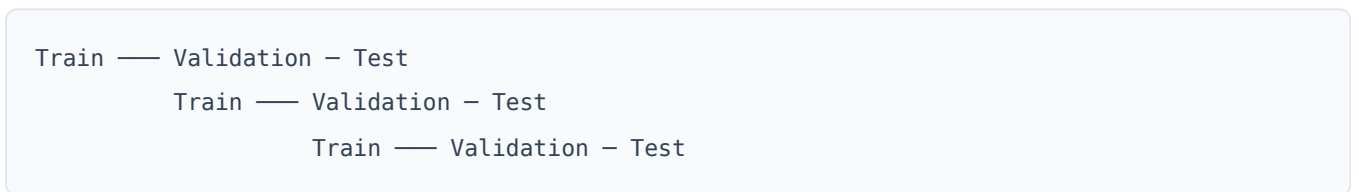
Ini harus menjadi **aturan absolut**.

Z.57. Walk-forward training

Saya menyarankan:



Kemudian window bergerak:



Ini disebut:

Walk Forward Validation.

Z.58. Contoh

Misalnya:

2021 – 01 → 2022 – 06

training.

2022 – 07 → 2022 – 09

validation.

2022 – 10 → 2022 – 12

test.

Kemudian window bergeser.

Z.59. Benchmark yang wajib

Kita jangan hanya membandingkan:

VQE

vs:

QAOA.

Kita harus membandingkan:

Buy&Hold

Equal Weight

Markowitz

Classical QUBO

VQE

QAOA

Dengan **identical data dan execution assumptions**.

Z.60. Metrics

Minimal:

Return

R_{total} .

Annualized return

R_{ann} .

Volatility

$$\sigma_{ann}.$$

Sharpe

$$Sharpe = \frac{R_{ann} - R_f}{\sigma_{ann}}.$$

Maximum drawdown

$$MDD = \max_t \frac{Peak_t - Equity_t}{Peak_t}.$$

Calmar

$$Calmar = \frac{R_{ann}}{MDD}.$$

Sortino

$$Sortino = \frac{R_{ann} - R_f}{\sigma_{down}}.$$

Z.61. Tetapi untuk sistem kita ada metrics tambahan

Karena tujuan kita bukan sekadar profit.

Kita harus mengukur:

$$Quantum Optimization Quality$$

misalnya:

$$Gap_{rel}.$$

Kemudian:

$$Feasibility Rate$$

berapa persen solusi memenuhi seluruh constraint.

Dan:

$$\text{Execution Efficiency.}$$

Misalnya:

$$\text{Implementation Shortfall.}$$

Z.62. Kita bahkan bisa membuat decomposition

Total performance:

$$P_{total}$$

dapat diuraikan menjadi:

$$P_{total} = P_{signal} + P_{allocation} + P_{execution} + P_{risk} - P_{cost}.$$

Ini sangat berguna untuk penelitian.

Jika hasil buruk:

Apakah prediction buruk?

atau:

QUBO buruk?

atau:

Execution buruk?

atau:

Risk management buruk?

Z.63. Quantum advantage tidak boleh didefinisikan sebagai "profit lebih tinggi"

Ini salah satu prinsip penelitian terpenting.

Quantum algorithm dapat memberikan:

$$E_Q \approx E^*$$

tetapi belum tentu:

$$Profit_Q > Profit_{classical}.$$

Karena market outcome bersifat stochastic.

Maka kita pisahkan:

Computational performance

$$Gap, Runtime, Qubits, Depth.$$

Financial performance

$$Return, Sharpe, MDD, Sortino.$$

Keduanya harus dilaporkan.

Z.64. Experimental matrix

Saya menyarankan eksperimen seperti:

Model	Optimizer	Data	Regime	Execution
Baseline	Equal Weight	OHLCV	No	Ideal
Markowitz	Classical	OHLCV	No	Realistic
Factor	Classical	OHLCV+Factors	Yes	Realistic
QUBO	Exact	Same	Yes	Realistic
QUBO	VQE	Same	Yes	Realistic
QUBO	QAOA	Same	Yes	Realistic

Ini jauh lebih kuat daripada sekadar:

"Saya menjalankan VQE pada crypto."

Z.65. Tahap Z menghasilkan satu master data flow

Pada akhirnya:

$$\begin{aligned}
 D_{0:t} &\rightarrow X_t \\
 &\rightarrow F_t \\
 &\rightarrow z_t \\
 &\rightarrow \hat{\mu}_t, \hat{\Sigma}_t \\
 &\rightarrow H_t(x) \\
 &\rightarrow x_t^* \\
 &\rightarrow w_t^* \\
 &\rightarrow q_t^* \\
 &\rightarrow TradePlan_t \\
 &\rightarrow Execution_t \\
 &\rightarrow Position_t \\
 &\rightarrow PnL_t \\
 &\rightarrow Evaluation_t.
 \end{aligned}$$

Z.66. Versi minimum viable system

Saya justru **tidak** ingin kita langsung membangun semua bagian.

Versi pertama seharusnya:

$$N = 8$$

assets.

$$\Delta t = 1h$$

decision.

Factors:

BTC, ETH, DXY, Gold, SPX, VIX

Forecast:

Factor Model + Rolling Covariance

Regime:

3 - state HMM

Optimization:

QUBO

Solver:

Exact + VQE + QAOA

Risk:

fixed fractional risk

Execution:

limit/market + spread + slippage

Backtest:

walk - forward

Z.67. Dan versi pertama sebaiknya belum menggunakan live trading

Urutannya:

Backtest → Paper Trading → Testnet → Small Capital → Production.

Bukan:

Backtest → Real Money.

Karena bahkan jika mathematical model benar, execution layer masih dapat mempunyai bug.

Z.68. Apa yang akan menjadi "core research question"?

Dengan blueprint ini, kita sekarang bisa mengajukan pertanyaan penelitian yang jauh lebih tajam:

Apakah quantum approximate optimization dapat menghasilkan solusi asset-selection yang kompetitif terhadap classical optimization pada short-horizon multi-asset trading dengan factor-conditioned risk, transaction cost, dan realistic execution constraints?

Kemudian subquestion:

Q1

Apakah QUBO menghasilkan solusi optimal/near-optimal?

Gap_{QUBO}.

Q2

Apakah VQE/QAOA mendekati solusi exact?

Gap_{quantum}.

Q3

Apakah solusi tersebut menghasilkan performa finansial yang lebih baik?

Sharpe, MDD, Return.

Q4

Apakah keuntungan bertahan setelah:

$$fees + spread + slippage?$$

Q5

Apakah model tetap robust ketika:

$$Regime_{train} \neq Regime_{test}?$$

Z.69. Ini juga membuat proyekmu jauh lebih dekat dengan quantitative research sebenarnya

Karena kita tidak lagi memiliki:

indicator $\rightarrow$ buy/sell.

Melainkan:

Information $\rightarrow$ Statistical Model $\rightarrow$ Conditional Distribution $\rightarrow$ Optimization Problem $\rightarrow$ Constrained Decision -
--



Dan **itu** adalah perbedaan besar antara trading bot sederhana dengan quantitative decision system.

Z.70. Arsitektur final Tahap A-Z

Kalau seluruh perjalanan kita diringkas:

A

|

└─ Market / financial foundation

|

B

└─ Return / risk / factor mathematics

|

C

└─ Cross-asset dependency

|

D

└─ Forecasting

|

E

└─ Regime

|

F

└─ Portfolio mathematics

|

G

└─ Risk

|

H

└─ Short-term trading

|

I

└─ Execution

|

J

└─ Position management

|

K

└─ Dynamic TP/SL

|

L

└─ Combinatorial optimization

|

M

└─ QUBO

|

N

└─ Ising



Dan sekarang kita mempunyai satu bentuk yang sangat jelas:

$$\boxed{F_t \xrightarrow{\Phi} (\hat{\mu}_t, \hat{\Sigma}_t, z_t) \xrightarrow{H} H_t \xrightarrow{Q} x_t^* \xrightarrow{\Psi} (w_t^*, q_t^*) \xrightarrow{\Omega} O_t \xrightarrow{F} s_{t+1}.$$

Inilah mathematical spine dari sistem yang kita bangun.

Langkah setelah Z

Menurut saya, setelah Z kita **jangan langsung membuat Tahap AA yang semakin abstrak**. Kita sudah cukup lama berada di level desain. Sekarang waktunya mulai mengubah blueprint menjadi **spesifikasi eksperimen yang benar-benar konkret**.

Jadi Tahap berikutnya yang paling masuk akal adalah:

AA — Definisi Dataset, Asset Universe, Timeframe, dan Walk-Forward Experiment

Di sana kita akan menentukan secara eksplisit, misalnya:

$$\text{Crypto Universe} = \{BTC, ETH, SOL, \dots\}$$
$$\text{Equity Universe} = \{AAPL, MSFT, NVDA, \dots\}$$
$$\text{Factors} = \{DXY, \text{Gold}, SPX, VIX, US10Y, BTC, ETH, \dots\}$$

lalu menentukan **tepat bagaimana data disinkronkan antara pasar crypto 24/7 dan saham yang hanya buka pada jam tertentu**, bagaimana missing data ditangani, bagaimana factor lag dibuat agar tidak terjadi look-ahead bias, bagaimana rolling window dihitung, serta bagaimana satu baris data pada t berubah menjadi satu QUBO H_t .

Dan menurut saya bagian sinkronisasi **crypto ↔ saham ↔ DXY ↔ Gold** ini justru sangat penting, karena di situlah desain kita akan mulai berhadapan dengan masalah nyata quantitative research, bukan sekadar matematika di atas kertas.